Хамилтонови обхождания

Дефиниция 1 *Нека $G = (V, E)$ е неориентиран свързан граф.*

Хамилтонов път *в G наричаме такъв път, който минава през всеки връх точно по веднъж. Ако началото и края на този път съвпадат, говорим за* Хамилтонов цикъл. *Графът G е* Хамилтонов, *ако в него съществува поне един Хамилтонов цикъл.*

Не е известен ефективен алгоритъм, който да проверява дали даден граф е Хамилтонов. Това е свързано с факта, че нямаме добра характеризация на Хамилтоновите графи, подобна на Ойлеровите графи. Всъщност има редица от *достатъчни* условия (но не и необходими) един граф да е Хамилтонов, например да съдържа $n \geq 3$ върха и степента на всеки връх да е поне $n/2$ или по-общо $d(u) + d(v) \geq n$ за всеки два несъседни върха u, v (последното се нарича теорема на Ore).

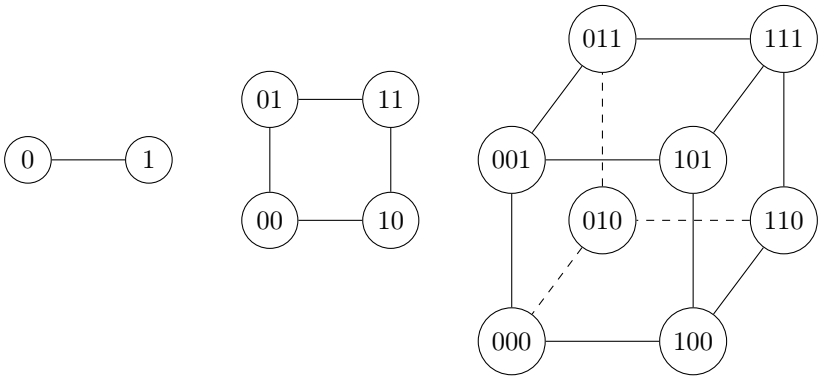
Ще завършим тази секция с пример за безкрайна редица от Хамилтонови графи.

Дефиниция 2 *Графът $\mathcal{B}_n = (\{0,1\}^n, E_n)$, където*

$$E_n = \{\{\alpha, \beta\} \mid \alpha, \beta \in \{0,1\}^n \text{ се различават в точно една позиция}\}$$

ще наричаме n-мерен двоичен куб *(съответно върховете му наричаме n-мерни двоични вектори).*

Например, при $n = 1, 2, 3$ получаваме следните графи:



Твърдение 3 *При $n \geq 2$ графът $\mathcal{B}_n$ е Хамилтонов.*

Доказателство. Индукция по n.

База: при $n = 2$ имаме Хамилтонов цикъл $00, 01, 11, 10, 00$.

Стъпка: нека $\alpha_0, \alpha_1, \ldots, \alpha_{2^n-1}, \alpha_0$ е Хамилтонов цикъл в $\mathcal{B}_n$. Да разгледаме следната последователност от върхове в $\mathcal{B}_{n+1}$:

$$0\alpha_0, 0\alpha_1, \ldots, 0\alpha_{2^n-1}, 1\alpha_{2^n-1}, 1\alpha_{2^n-2}, \ldots, 1\alpha_0, 0\alpha_0.$$

Лесно се вижда, че всеки два съседни върха се различават точно в една позиция, т.е. това е цикъл в $\mathcal{B}_{n+1}$ и освен това всеки връх от $\{0,1\}^{n+1}$ (с изключение на началото) участва еднократно. Казано по друг начин, това е Хамилтонов цикъл в $\mathcal{B}_{n+1}$.

Използваният метод за намиране на Хамилтонов цикъл в $\mathcal{B}_n$ се нарича *отражение*.

Накрая да отбележим, че при $n = 1$ в $\mathcal{B}_1$ няма Хамилтонов цикъл, но има Хамилтонов път.

Минимални пътища в графи с тегла по ребрата

Нека $G = (V, E)$ е свързан граф и да предположим, че е зададена функция $c : E \to \mathbb{R}$. За всяко ребро $e \in E$ реалното число $c(e)$ ще наричаме *цена или тегло* на реброто e.

При даден път $p = v_0, v_1, \ldots, v_k$, *цена или тегло на пътя* p наричаме реалното число $c(p) = \sum_{j<k} c(\{v_j, v_{j+1}\})$, т.е. сумата от цените/теглата на ребрата, участващи в пътя.

За целите на тази секция ще предполагаме, че $c(e) > 0$ за всяко ребро e.

Дефинираме функцията $dist : V \times V \to \mathbb{R}^{\geq 0}$ по следния начин:

$$dist(u, v) \;=\; \min\{c(p) \mid p \text{ е път с начало } u \text{ и край } v\}.$$

По този начин, $dist(u, v)$ е най-малката възможна цена на път с начало u и край v. Път p от u до v, такъв че $c(p) = dist(u, v)$ наричаме *минимален път* от u до v.

Забележете, че всеки минимален път от u до v е прост (това следва от факта, че теглата на ребрата са положителни). В частност, $dist$ е добре дефинирана функция, тъй като простите пътища от u до v са краен брой.

Ако в графа допускахме ребра с отрицателни тегла, то функцията $dist$ нямаше да е добре дефинирана. Причината е, че в такъв случай между никои два върха няма път с минимална цена: можем да вървим последователно напред и назад по някое отрицателно ребро (и то е достижимо от всички върхове), като така ще получаваме все по-малка цена на пътя.

Ако всички ребра в графа имат цена 1, то функцията $dist$ е същата, която разгледахме при обхождането в ширина.

Отново важи *принципа за оптималност*: ако p е минимален път, то всяка част от p също е минимален път.

Също е вярно *неравенството на триъгълника*:

$$dist(u, v) \leq dist(u, w) + dist(w, v).$$

Формулираме *основната задача* в тази тема: Даден е свързан граф $G = (V, E)$ с тегла по ребрата $c : E \to \mathbb{R}^{>0}$, както и начален връх v_0. Да се намерят (прости) минимални пътища от v_0 до всички върхове на V.

В частния случай, когато $c(e)$ има една и съща стойност за всички ребра $e \in E$, задачата се решава с обхождане в ширина с начален връх v_0.

За общия случай ще разгледаме *алгоритъма на Дайкстра*.

В него съществена роля ще играят два масива d и $pred$. В масива d ще съхраняваме приближения на цените на търсените минимални пътища. На всяка итерация на основния цикъл ще намираме минимален път от v_0 до един нов връх u_0. В края на алгоритъма, масивът d ще съдържа коректните цени на минималните пътища. Тези пътища ще образуват дърво с корен v_0, което ще представим със списък на бащите в масива $pred$.

Инициализация: $d[v_0] = 0$; $d[v] = \infty$ за $v \neq v_0$; $U = \emptyset$;
Основен цикъл на алгоритъма:
  докато $U \neq V$,
    избираме $u_0 \notin U$, за който $d[u_0]$ е минимално;
    добавяме u_0 към U: $U = U \cup \{u_0\}$;
    за всички съседи u на u_0:
      ако $d[u] > d[u_0] + c(\{u_0, u\})$, то
        { $d[u] = d[u_0] + c(\{u_0, u\})$; $pred[u] = u_0$; }

Считаме, че ∞ е достатъчно голяма стойност (например, по-голяма от сумата на всички ребра в графа). Забележете, че промяна в стойността на $d[u]$ може да се извърши само с по-малка стойност (в някои учебници тази промяна се нарича *релаксация*).

Лема 4 *На всяка итерация на основния цикъл е изпълнено:* $dist(v_0, v) \leq d[v]$ *за всеки връх* $v \in V$.

Доказателство. Индукция по броя на итерациите.

База: Преди да започне изпълнението на цикъла имаме $dist(v_0, v_0) = 0 = d[v_0]$ и $dist(v_0, v) < \infty = d[v]$ за $v \neq v_0$.

Стъпка: Нека след някакъв брой итерации е изпълнено $dist(v_0, v) \leq d[v]$ за всички $v \in V$. На следващата итерация промяна в стойностите на d може да има само в $d[u]$ за някой съсед u на u_0, като новата стойност $d[u]$ е $d[u_0] + c(\{u_0, u\})$. В такъв случай:

$$\begin{aligned} dist(v_0, u) &\leq dist(v_0, u_0) + dist(u_0, u) \\ &\leq dist(v_0, u_0) + c(\{u_0, u\}) \leq d[u_0] + c(\{u_0, u\}) = d[u]. \end{aligned}$$

Първо сме използвали неравенството на триъгълника, след това очевидния факт $dist(u_0, u) \leq c(\{u_0, u\})$ и накрая индукционното предположение. Разбира се, за стойностите на $d[v]$, които не се променят и след новата итерация имаме $dist(v_0, v) \leq d[v]$.

Лема 5 *За произволен връх $v \in V$, ако на някоя итерация е изпълнено $dist(v_0, v) = d[v]$, то това равенство се запазва и на следващите итерации до края на алгоритъма.*

Доказателство. От първата лема е изпълнено $dist(v_0, v) \leq d[v]$ на всяка итерация. Както вече отбелязахме, при актуализиране стойността $d[v]$ може само да намалява. По този начин, ако $d[v]$ достигне стойност $dist(v_0, v)$ на някоя итерация тя повече няма да се променя.

Теорема 6 (коректност на алгоритъма на Дайкстра) *След завършването на основния цикъл на алгоритъма на Дайкстра, за всеки връх $v \in V$ е изпълнено $dist(v_0, v) = d[v]$.*

Доказателство. Достатъчно е да покажем, че на всяка итерация в момента на избор на u_0 е изпълнено $dist(v_0, u_0) = d[u_0]$. Наистина, тъй като алгоритъмът приключва при $U = V$, всеки връх трябва да бъде избран на някоя итерация, т.е. да бъде u_0 на някоя итерация и от втората лема равенството $dist(v_0, u_0) = d[u_0]$ остава вярно до края на алгоритъма.

Допускаме противното, т.е. че алгоритъмът греши и нека u_0 е *първият* избран връх, такъв че $dist(v_0, u_0) \neq d[u_0]$ в момента на неговия избор. Разбира се, това означава, че $u_0 \neq v_0$ и също така от първата лема е изпълнено $dist(v_0, u_0) < d[u_0]$. Да изберем минимален път p от v_0 до u_0. В разглеждания момент при избора на u_0 имаме $u_0 \notin U$ и $v_0 \in U$, тъй като v_0 се избира на първата итерация. По този начин, p има за начало връх от U и за край връх извън U, така че съществува ребро $\{x, y\}$ от пътя p, такова че $x \in U$ и $y \notin U$. С други думи, пътят p изглежда така:

$$v_0, \ \ldots\ldots, \ \underbrace{x}_{\in U}, \ \underbrace{y}_{\notin U}, \ \ldots\ldots, \ u_0.$$

Разбира се, възможно е $v_0 = x$ или $u_0 = y$. Тъй като $x \in U$, имаме че x е избран на предходна итерация (преди u_0) и от избора на u_0 получаваме, че на тази предходна итерация е изпълнено $dist(v_0, x) = d[x]$. На същата итерация $d[y]$ ще получи стойност $d(v_0, y)$, тъй като p е минимален път и

$$dist(v_0, y) \ = \ dist(v_0, x) + c(\{x, y\}) \ = \ d[x] + c(\{x, y\}).$$

От втората лема, тази стойност на $d[y]$ се запазва и по-нататък, докато се върнем на оригиналната итерация при избора на u_0. Но тогава имаме:

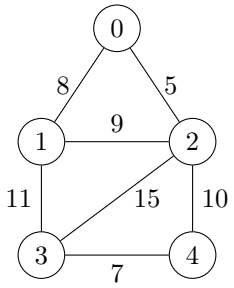
$$d[y] \ = \ dist(v_0, y) \ \leq \ dist(v_0, u_0) \ < \ d[u_0],$$

като за първото неравенство използваме, че цените на ребрата са положителни. Достигнахме до противоречие с избора на u_0, като връх извън U с минимална стойност на d.

Следствие 7 *След изпълнението на алгоритъма, pred задава списък на бащите на дърво на минимални пътища с корен v_0.*

Доказателство. На всяка итерация, $pred$ задава списък на бащите на дърво с корен v_0, което съдържа върховете v с $d[v] \neq \infty$ и освен това единственият прост път от v_0 до v в това дърво има цена $d[v]$ (забележете, че ако стойността $d[u]$ се променя на някоя итерация, то $u \notin U$ на тази итерация и затова върхът u не може да има наследници в текущото дърво). В края на алгоритъма $dist(v_0, v) = d[v]$ за всеки връх v, така че $pred$ задава дърво на минимални пътища с корен v_0, което съдържа всички върхове.

Ще изпълним алгоритъма на Дайкстра върху следния примерен граф:



Нека началният връх да бъде 0.

Основният цикъл ще направи 5 итерации. Стойностите в масивите d и $pred$ се изменят така:

$$d = (\underline{0},\ \infty,\ \infty,\ \infty,\ \infty), \qquad pred = (-,\ -,\ -,\ -,\ -)$$
$$d = (0,\ 8,\ \underline{5},\ \infty,\ \infty), \qquad pred = (-,\ 0,\ 0,\ -,\ -)$$
$$d = (0,\ \underline{8},\ 5,\ 20,\ 15), \qquad pred = (-,\ 0,\ 0,\ 2,\ 2)$$
$$d = (0,\ 8,\ 5,\ 19,\ \underline{15}), \qquad pred = (-,\ 0,\ 0,\ 1,\ 2)$$
$$d = (0,\ 8,\ 5,\ \underline{19},\ 15), \qquad pred = (-,\ 0,\ 0,\ 1,\ 2)$$

Подчертаните стойности показват кой връх се избира да влезе в множеството U на съответната итерация.

На първата итерация се избира началният връх 0 и масивът d се актуализира в неговите съседи $1, 2$.

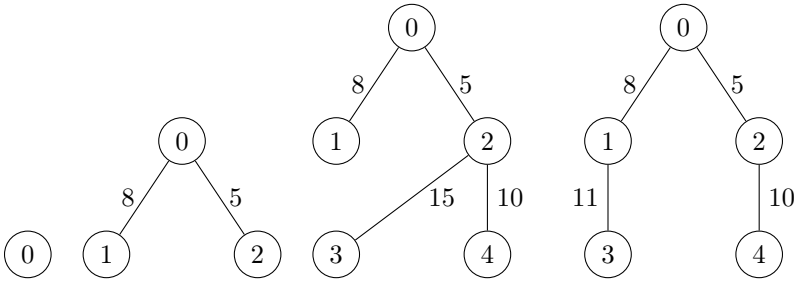
На втората итерация се избира върхът 2. Пътят до 1 през 2 има цена 14, така че $d[1]$ не се променя. Върховете 3 и 4 се достигат за първи път на тази итерация и пътищата им през 2 имат цена $d[3] = 20$ и $d[4] = 15$.

На третата итерация се избира върхът 1. Пътят до 3 през 1 има цена 19, така че $d[3]$ става 19 и съответно $pred[3]$ се променя на 1.

На четвъртата итерация се избира върхът 4. Пътят до 3 през 4 има цена 22, така че $d[3]$ не се променя.

На последната пета итерация остава да се избере върхът 3. Разбира се, тук няма повече промени на стойности.

Дървото на текущите минимални пътища претърпява следните последователни промени:



За домашно изпълнете алгоритъма на Дайкстра и с другите възможности за начални върхове $1, 2, 3, 4$.

Оптимални покриващи дървета

Нека $G = (V, E)$ е свързан граф и $c : E \to \mathbb{R}$ е функция, задаваща цени (тегла) по ребрата.

За покриващо дърво $D = (V, E')$ на G дефинираме *цена (тегло) на дървото* D да бъде реалното число $c(D) = \sum_{e \in E'} c(e)$, т.е. сумата на цените на всички ребра, участващи в D.

Дефиниция 8 *Покриващото дърво* D_0 *наричаме* минимално, *ако* $c(D_0) \leq c(D)$ *за всяко покриващо дърво* D *на* G.
Покриващото дърво D_0 *наричаме* максимално, *ако* $c(D_0) \geq c(D)$ *за всяко покриващо дърво* D *на* G.

Лесно се съобразява, че такива дърветa съществуват: графът G има краен брой покриващи дърветa и във всяко крайно множество от реални числа има най-малко и най-голямо число.

За разлика от алгоритъма на Дайкстра, тук няма изискване цените на ребрата да са положителни. Ще разгледаме алгоритъм, който намира минимално покриващо дърво. С негова помощ може да се намери и максимално покриващо дърво, ако първоначално се обърнат знаците на цените на всички ребра в графа (или ако "минимална" се замени с "максимална" по-долу в алгоритъма).

Задача: Даден е свързан граф $G = (V, E)$ с тегла по ребрата $c : E \to \mathbb{R}$. Да се намери минимално покриващо дърво $D = (V, E')$ на графа G.

Разглеждаме *алгоритъма на Прим*.

Идеята в алгоритъма е да присъединяваме върховете един след друг по такъв начин, че цената на присъединяващото ребро на всяка стъпка да бъде възможно най-малка. Имаме свобода да започнем с произволен връх.

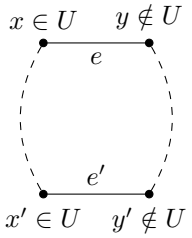
Инициализация: избираме $r \in V,\ U = \{r\};\ E' = \emptyset$;
Основен цикъл на алгоритъма:
докато $U \neq V$,
нека $e = \{x, y\}$ е ребро, за което $x \in U,\ y \notin U$ и $c(\{x, y\})$ е минимална;
присъединяваме y към x в U: $U = U \cup \{y\};\ E' = E' \cup \{e\}$;

Теорема 9 (коректност на алгоритъма на Прим) *След изпълнението на алгоритъма, $D = (V, E')$ е минимално покриващо дърво на графа G.*

Доказателство. Ще докажем, че на всяка итерация на основния цикъл съществува минимално покриващо дърво, което разширява дървото (U, E'). Разбира се, това ще е изпълнено и в края на алгоритъма: има минимално покриващо дърво, разширяващо дървото (V, E'), но това може да се случи само ако двете дървета съвпадат (върховете им са едни и същи).

База: В началото $(U, E') = (\{r\}, \emptyset)$, така че произволно минимално покриващо дърво върши работа (поне едно съществува, както отбелязохме).

Стъпка: Нека $D_0 = (V, E_0)$ е минимално покриващо дърво, което разширява (U, E') $(E' \subseteq E_0)$ на някоя итерация (това е индукционното предположение). Нека $e = \{x, y\}$ е реброто, което е избрано на тази итерация. Лесният случай е, когато $e \in E_0$, т.е. когато реброто e участва в минималното покриващо дърво D_0. Тогава D_0 разширява $(U \cup \{y\}, E' \cup \{e\})$ и сме готови. Да разгледаме другия случай: $e \notin E_0$. Добавяме реброто e към минималното покриващо дърво D_0. Знаем, че тогава се получава единствен цикъл, в който участва e. Цикълът започва с $x \in U$ и стига до $y \notin U$, преди да се завърне в x. За да стане това, със сигурност има друго ребро $e' = \{x', y'\}$, такова че $x' \in U,\ y' \notin U$:

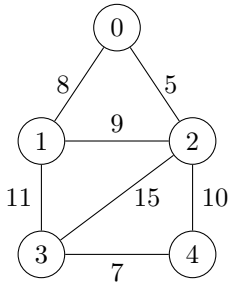


За реброто e' имаме $e' \in E_0,\ e' \notin E'$. От избора на e в алгоритъма, имаме неравенството $c(e) \leq c(e')$. Да образуваме $D_0' = (V, E_0 \setminus \{e'\} \cup \{e\})$. Ясно е, че D_0' също е покриващо дърво на G. За неговата цена имаме:

$$c(D_0') \ = \ c(D_0) - c(e') + c(e) \ \leq \ c(D_0).$$

Тъй като дървото D_0 е минимално, D_0' също е минимално (всъщност дори имаме равенство и $c(e) = c(e')$). Тъй като $E' \cup \{e\} \subseteq E_0 \setminus \{e'\} \cup \{e\}$, дървото D_0' разширява $(U \cup \{y\}, E' \cup \{e\})$, така че доказателството е завършено.

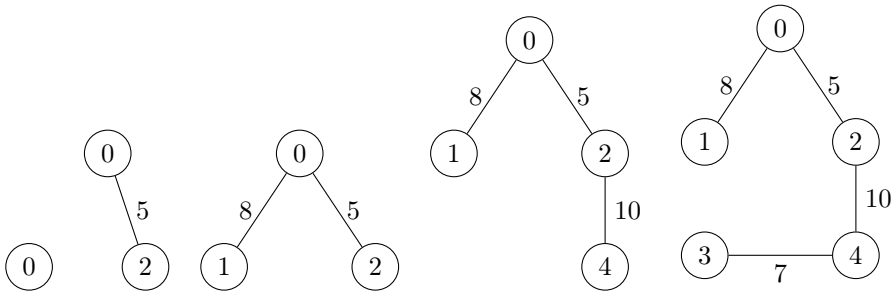
Да изпълним алгоритъма на Прим върху същия примерен граф от по-горе:



Нека началният връх е 0. Множеството U се променя по следния начин:

$$U = \{0\},\ \ U = \{0, 2\}\ (e = \{0, 2\}),\ \ U = \{0, 1, 2\}\ (e = \{0, 1\}),$$
$$U = \{0, 1, 2, 4\}\ (e = \{2, 4\}),\ \ U = \{0, 1, 2, 3, 4\}\ (e = \{4, 3\}),$$

като постепенно се получава минималното покриващо дърво:



Може да проверите, че за примерния граф никое от дърветата, получени с алгоритъма на Дайкстра не съвпада с минималното покриващо дърво, получено с алгоритъма на Прим. Въпреки, че има сходство в двата алгоритъма, при Дайкстра разширението става по "локален" критерий (отдалеченост от фиксираното начало v_0), докато при Прим разширението е по "глобален" критерий (отдалеченост от всички добавени до момента върхове).